Estudo de caso com *Interface*

Estudo de Caso: criando e utilizando interfaces

- Suponha que a empresa do exemplo de folha de pagamento deseja realizar operações de contabilidade em um único aplicativo de contas a pagar (calcular vencimentos de funcionarios também calcula pagamento devido a cada fatura – contas de mercadorias adquiridas).
- Poderíamos calcular coisas tão diferentes polimorficamente?

Estudo de Caso: criando e utilizando interfaces

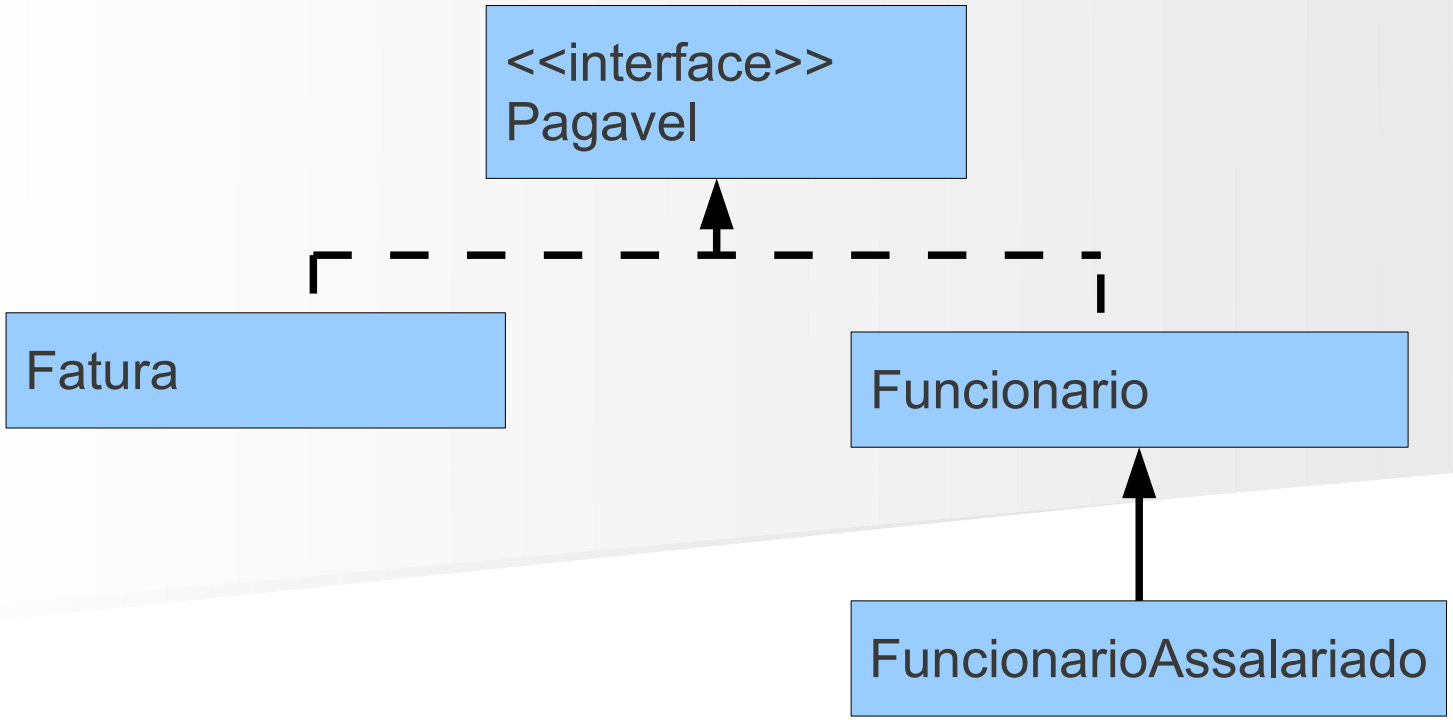
- Suponha que a empresa do exemplo de folha de pagamento deseja realizar operações de contabilidade em um único aplicativo de contas a pagar (calcular vencimentos de funcionarios também calcula pagamento devido a cada fatura – contas de mercadorias adquiridas).
- Poderíamos calcular coisas tão diferentes polimorficamente?
- As interfaces do Java oferecem essa capacidade de exigir que classes não relacionadas implementem um conjunto de métodos comuns
  - por exemplo o método que calcula a quantia de um pagamento.

Estudo de Caso: criando e utilizando interfaces

- Para utilizar uma interface, a classe concreta deve especificar que implementa (**implements**) a interface e deve declarar todos os métodos da interface
- Vamos criar a interface *Pagavel*
  - Método getValorPagamento() que retorna um valor double a ser pago para um objeto de qualquer classe que implementa a interface
  - Este método é uma versão geral do método ganhos() do exemplo que usamos classe abstrata
  - Obs: ao declarar um método em uma interface escolha um nome de método que descreva o propósito do método de forma geral pois pode ser implementado por muitas classes não relacionadas

Estudo de Caso: criando e utilizando interfaces

- A classe Funcionario é modificada para implementar a interface *Pagavel* e apresentamos uma classe Fatura que também implementa a mesma interface.



Estudo de Caso: criando e utilizando interfaces

- Obs:
  - Métodos de Interface sempre são public e abstract e escrever isso é redundante
  - Campos em uma interface sempre são final e static e também é redundância escrever

```
public interface Pagavel
{
    double getValorPagamento();//calcula pagamento;
    //nenhuma implementacao
}
```

Estudo de Caso: criando e utilizando interfaces

- Modificando a classe Funcionario
  - Inserimos implements Pagavel
  - A classe não implementa o método e deve ser mantida abstrata para evitar erro de compilacao

```
//superclasse abstrata Funcionario
public abstract class Funcionario implements Pagavel
{
    private String nome;
    private String sobrenome;
    private String id;
```

Estudo de Caso: criando e utilizando interfaces

- Modificando a classe FuncionarioAssalariado
  - Apos implementar Pagavel devemos renomear o metodo ganhos para getValorPagamento nas subclasses de Funcionario

```
//sobrescreve o metodo abstrato
public double getValorPagamento(){
    return getSalarioSemanal();
}
```

Estudo de Caso: criando e utilizando interfaces

```
//classe Fatura que implementa interface Pagavel
public class Fatura implements Pagavel
{
    private String numero;
    private String descricao;
    private int quantidade;
    private double precoItem;
    //construtor com quatro argumentos
    public Fatura(String numero, String descricao,
        int quantidade, double precoItem){
        this.numero = numero;
        this.descricao = descricao;
        setQuantidade(quantidade);//valida e armazena
        setPrecoItem(precoItem);
    }
```

Estudo de Caso: criando e utilizando interfaces

```
//configura numero da peca
public void setNumero(String numero){
    this.numero = numero;
}
//retorna numero da peca
public String getNumero(){
    return numero;
}
//configura descricao da peca
public void setDescricao(String descricao){
    this.descricao = descricao;
}
//retorna descricao
public String getDescricao(){
    return descricao;
}
```

Estudo de Caso: criando e utilizando interfaces

```
//valida e configura quantidade
public void setQuantidade(int quantidade){
    this.quantidade = (quantidade < 0) ? 0 : quantidade;
}
//retorna quantidade
public int getQuantidade(){
    return quantidade;
}
//valida e configura preco por item
public void setPrecoItem(double preco){
    this.precoItem = (preco < 0.0) ? 0.0 : preco;
}
//retorna preco por item
public double getPrecoItem(){
    return precoItem;
}
```

Estudo de Caso: criando e utilizando interfaces

```
    //retorna representacao string do objeto
    public String toString(){
        return String.format("%s: \n%s: %s(%s) \n%s: %d \n%s: R$%,.2f",
            "fatura", "numero da peca", getNumero(), getDescricao(),
            "quantidade", getQuantidade(), "preco por item",
            getPrecoItem());
    }
    //metodo requerido pela interface Pagavel
    public double getValorPagamento(){
        return getQuantidade()*getPrecoItem();//calcula custo total
    }
}
```

Estudo de Caso: criando e utilizando interfaces

- Quando uma classe implementa uma interface a mesma relação *é um* da herança se aplica
- Podemos dizer que:
  - Funcionario é um Pagavel
  - FuncionárioAssalariado é um Pagável
  - Fatura é um Pagavel

Estudo de Caso: criando e utilizando interfaces

- Utilizando a interface Pagavel objetos de classes completamente desconectadas (Faturas e Funcionarios) podem ser processados polimorficamente
- Crie uma classe de Teste

Estudo de Caso: criando e utilizando interfaces

```
public class TesteInterfacePagavel
{
    public static void main(String[] args){
        //cria array de quatro elementos Pagavel
        Pagavel[] objetosPagaveis = new Pagavel[4];

        //preenche o array com objetos que implementam
        //a interface pagavel
        objetosPagaveis[0] = new Fatura("01234", "banco", 2, 375.00);
        objetosPagaveis[1] = new Fatura("56789", "pneu", 4, 79.95);
        objetosPagaveis[2] = new FuncionarioAssalariado(
            "Joao", "da Silva", "111-111-111", 800.00);
        objetosPagaveis[3] = new FuncionarioAssalariado(
            "Maria", "Souza", "888-888-888", 1200.00);
```

Estudo de Caso: criando e utilizando interfaces

```
System.out.println("Faturas e funcionarios processados polimorficamente:\n");


//processa genericamente cada elemento no array
for(Pagavel pagAtual : objetosPagaveis){
    //gera saida do objeto atual
    System.out.printf("%s \n%s: R$%,.2f\n\n", pagAtual.toString(),
        "valor pagamento", pagAtual.getValorPagamento());
}
```

```
Faturas e funcionarios processados polimorficamente:

fatura:
numero da peca: 01234(banco)
quantidade: 2
preco por item: R$375,00
valor pagamento: R$750,00

fatura:
numero da peca: 56789(pneu)
quantidade: 4
preco por item: R$79,95
valor pagamento: R$319,80

Funcionario Assalariado: Joao da Silva
Identidade: 111-111-111
salario semanal: : R$800,00
valor pagamento: R$800,00

Funcionario Assalariado: Maria Souza
Identidade: 888-888-888
salario semanal: : R$1.200,00
valor pagamento: R$1.200,00
```

Interfaces comuns da Java API

- Permitem que você use suas próprias classes dentro das estruturas fornecidas pelo java.
  - Comparable
    - permite que objetos que a impelementam sejam comparados entre si
  - Serializable
    - utilizada para identificar classes cujos objetos podem ser gravados (serializados) ou lidos

Interfaces comuns da Java API

- Runnable
  - Implementada por qualquer classe da qual objetos devem ser capazes de executar em paralelo
- SwingConstants
  - Contem um conjunto de constantes utilizadas em programação de interfaces graficas com usuários (GUIs) para posicionar elementos na tela

Exercícios